

Actividad 9: Programación Dinámica

Curso: Análisis y Diseño de Algoritmos

Repositorio GitHub: [Enlace al Repositorio Aquí](#)

1. Problema de las Escaleras

1.1. Explicación del Algoritmo

El problema requiere calcular las formas diferentes de subir una escalera de N escalones avanzando de a 1 o 2 pasos. Al tener subestructura óptima, para llegar al escalón i , obligatoriamente se tuvo que dar un paso desde el escalón $i - 1$ o desde el escalón $i - 2$. Por tanto, las formas de llegar a i son la suma de las formas de llegar a sus dos escalones predecesores (similar a la sucesión de Fibonacci).

1.2. Estado de la Programación Dinámica

Se utiliza un arreglo unidimensional dp , donde:

$$dp[i] = \text{Número de formas distintas de llegar al escalón } i.$$

1.3. Relación de Recurrencia

Para cualquier escalón $i \geq 2$:

$$dp[i] = dp[i - 1] + dp[i - 2]$$

1.4. Casos Base

Los casos base corresponden a situaciones donde no es necesario realizar saltos o la escalera es muy corta:

- $dp[0] = 1$: Representa el estado inicial (estar en el suelo). Hay una única forma de estar ahí: no dar ningún paso.
- $dp[1] = 1$: Solo hay una forma de llegar al primer escalón: dando un paso de 1 escalón desde el suelo.

1.5. Análisis de Complejidad

- **Complejidad Temporal $\mathcal{O}(N)$** : Se ejecuta un ciclo lineal único que va desde el escalón 2 hasta el escalón N . En cada iteración se realiza una suma básica en tiempo constante $\mathcal{O}(1)$. Por lo tanto, el tiempo de ejecución crece de manera directamente proporcional al número de escalones N .
- **Complejidad Espacial $\mathcal{O}(N)$** : El algoritmo requiere instanciar una tabla (arreglo unidimensional) de tamaño $N + 1$ para almacenar los resultados del estado DP en cada posición intermedia. Esto implica un uso de memoria que escala linealmente en función de N .

1.6. Código Fuente

A continuación se muestra la implementación completa del algoritmo en Java, incluyendo el menú interactivo para el usuario:

```
1 import java.util.Arrays;
2 import java.util.Scanner;
3
4 public class Escaleras {
5
6     public static void print(Object msg) {
7         System.out.print(msg);
8     }
9
10    public static void println(Object msg) {
11        System.out.println(msg);
12    }
13
14    public static long[] calcularEscaleras(int n) {
15        if (n == 0) {
16            return new long[]{1};
17        }
18
19        long[] dp = new long[n + 1];
20
21        dp[0] = 1;
22        dp[1] = 1;
23
24        for (int i = 2; i <= n; i++) {
25            dp[i] = dp[i - 1] + dp[i - 2];
26        }
27
28        return dp;
29    }
30
31    public static void ejecutarCaso(int n, String numeroCaso) {
32        println("-----");
33        println("Caso de prueba " + numeroCaso + ": N = " + n);
34
35        long[] dp = calcularEscaleras(n);
36        long formasPosibles = dp[n];
37
38        println("Formas posibles: " + formasPosibles);
39        print("Tabla DP: ");
40        println(Arrays.toString(dp));
41    }
42
43    public static void main(String[] args) {
44        println(" SOLUCION: Problema de las Escaleras (PD)");
45
46        ejecutarCaso(5, "1");
47
48        ejecutarCaso(7, "2");
49
50        Scanner scanner = new Scanner(System.in);
51        try {
52            print("ingrese el numero de escalones N que desea analizar: ");
53
54            int nUsuario = scanner.nextInt();
55
56            if (nUsuario < 0) {
57                println("El numero de escalones debe ser un valor no negativo.");
58            } else {
59                ejecutarCaso(nUsuario, "Usuario");
60            }
61
62        } catch (Exception e) {
63            println("Error: ingresar unicamente numeros enteros");
64        } finally {
65            scanner.close();
66        }
67    }
68 }
```

Listing 1: Implementación en Java del Problema de las Escaleras

1.7. Evidencias de Ejecución

La implementación se validó ejecutando casos de prueba y permitiendo una entrada interactiva. A continuación se incluye la captura de pantalla con los resultados arrojados por consola.

```

diego {at-symbol-color} archlinux
^ OS      -> Arch Linux x86_64
^ Kernel  -> Linux 7.0.13-arch1-1
^ Uptime  -> 7 mins
^ PKGS    -> 11 (Flatpak), 1145 (pacman)

^ WM      -> Hyprland 0.55.4 (Wayland)
^ Shell   -> zsh 5.9.1
^ Term    -> kitty 0.47.1
^ Font    -> Fira Code Medium (14pt, Regular) [Qt], Fira Code (14pt, SemiBold) [GTK3]

^ CPU     -> Intel(R) Core(TM) i5-8265U (8) @ 3.90 GHz
^ RAM     -> 5.57 GiB / 7.68 GiB (73%)
^ Bateria -> 100% [AC Connected]

diego@archlinux in ~
$ cd ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09 on m
ain x (origin/main)
$ java Escaleras.java
SOLUCION: Problema de las Escaleras (PD)
-----
Caso de prueba 1: N = 5
Formas posibles: 8
Tabla DP: [1, 1, 2, 3, 5, 8]
-----
Caso de prueba 2: N = 7
Formas posibles: 21
Tabla DP: [1, 1, 2, 3, 5, 8, 13, 21]
ingrese el numero de escalones N que desea analizar: 8
-----
Caso de prueba Usuario: N = 8
Formas posibles: 34
Tabla DP: [1, 1, 2, 3, 5, 8, 13, 21, 34]
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09 on m
ain x (origin/main)
$

```

Figura 1: Captura de pantalla de la ejecución mostrando la tabla DP [1, 1, 2, 3, 5, 8] para N=5.

2. Problema del Cambio Mínimo de Monedas

2.1. Explicación del Algoritmo

El objetivo es encontrar la cantidad mínima de monedas necesarias para sumar una cantidad C , disponiendo de un conjunto de M denominaciones de monedas. A diferencia del enfoque ávido (*greedy*), el cual puede fallar o retornar soluciones subóptimas para sistemas de monedas no canónicos, la Programación Dinámica garantiza la obtención de la solución óptima global.

Utilizamos un enfoque Bottom-Up de abajo hacia arriba. Construimos las soluciones óptimas para todos los montos intermedios i desde 1 hasta la cantidad objetivo C . Para cada monto i , iteramos a través del conjunto de denominaciones y evaluamos si tomar una moneda de valor *moneda* nos da una solución mejor que la registrada previamente. Además, para la reconstrucción del camino óptimo, se registra qué denominación de moneda fue elegida para cada monto en una tabla auxiliar.

2.2. Estado de la Programación Dinámica

Definimos el estado de la programación dinámica en una tabla dp de tamaño $C + 1$, donde:

$$dp[i] = \text{Cantidad mínima de monedas necesarias para formar el monto } i.$$

2.3. Relación de Recurrencia

Para cada monto intermedio i (desde 1 hasta C), la relación de actualización es:

$$dp[i] = \min_{moneda \in \text{Monedas}, moneda \leq i} (dp[i], dp[i - moneda] + 1)$$

2.4. Casos Base

Los límites iniciales se establecen de la siguiente manera:

- $dp[0] = 0$: Se requieren exactamente cero monedas para representar un monto de valor 0.
- $dp[i] = \infty$ para todo $i > 0$: Inicialmente asumimos que todos los montos mayores a 0 son inaccesibles, representados con un valor infinito práctico (`Integer.MAX_VALUE` en Java, controlando el desbordamiento de enteros al sumar).

2.5. Análisis de Complejidad

- **Complejidad Temporal** $\mathcal{O}(M \cdot C)$: El algoritmo cuenta con dos ciclos anidados. El ciclo externo recorre todos los montos intermedios i de 1 a C , y el ciclo interno evalúa las M denominaciones de monedas disponibles. Dado que se realizan operaciones aritméticas y comparaciones elementales $\mathcal{O}(1)$ dentro de los ciclos, el tiempo total es proporcional a la multiplicación de ambas variables.
- **Complejidad Espacial** $\mathcal{O}(C)$: Se utilizan dos arreglos de tamaño $C + 1$: el primero (`dp`) almacena los montos mínimos y el segundo (`monedaUsada`) guarda el valor de la moneda elegida para la posterior reconstrucción de la combinación. El espacio total es lineal en relación al monto C .

2.6. Código Fuente

A continuación se presenta la implementación del algoritmo de Cambio de Monedas en Java, la cual incorpora la lógica de reconstrucción de la combinación óptima de monedas empleada:

```

1 import java.util.Arrays;
2 import java.util.Scanner;
3
4 public class CambioMonedas {
5
6     private static final int INFINITO = Integer.MAX_VALUE;
7
8     public static void print(Object msg) {
9         System.out.print(msg);
10    }
11
12    public static void println(Object msg) {
13        System.out.println(msg);
14    }
15
16    private static class ResultadoDP {
17        int[] dp;
18        int[] monedaUsada;
19    }
20
21    private static ResultadoDP calcularDP(int[] monedas, int cantidad) {
22        int[] dp = new int[cantidad + 1];
23        int[] monedaUsada = new int[cantidad + 1];
24
25        Arrays.fill(dp, INFINITO);
26        dp[0] = 0;
27
28        for (int i = 1; i <= cantidad; i++) {
29            for (int moneda : monedas) {
30                if (moneda <= i && dp[i - moneda] != INFINITO) {
31                    int candidato = dp[i - moneda] + 1;
32                    if (candidato < dp[i]) {
33                        dp[i] = candidato;
34                        monedaUsada[i] = moneda;
35                    }
36                }
37            }
38        }
39
40        ResultadoDP resultado = new ResultadoDP();
41        resultado.dp = dp;
42        resultado.monedaUsada = monedaUsada;
43        return resultado;
44    }
45
46    private static String reconstruirCombinacion(int[] monedaUsada, int cantidad) {
47        StringBuilder combinacion = new StringBuilder();
48        int restante = cantidad;
49
50        while (restante > 0) {
51            int moneda = monedaUsada[restante];
52            if (combinacion.length() > 0) {
53                combinacion.append("+");
54            }
55            combinacion.append(moneda);
56            restante -= moneda;
57        }
58
59        return combinacion.toString();
60    }
61
62    public static void ejecutarCaso(int[] monedas, int cantidad, String numeroCaso) {
63        println("-----");
64        println("Caso de prueba " + numeroCaso + ": Monedas = "
65            + Arrays.toString(monedas) + ", Cantidad = " + cantidad);
66
67        ResultadoDP resultado = calcularDP(monedas, cantidad);
68
69        if (resultado.dp[cantidad] == INFINITO) {
70            println("No es posible formar la cantidad solicitada con las monedas dadas.");
71            return;
72        }
73    }

```

```

74     int minimoMonedas = resultado.dp[cantidad];
75     String combinacion = reconstruirCombinacion(resultado.monedaUsada, cantidad);
76
77     println("Cantidad minima de monedas: " + minimoMonedas);
78     println("Combinacion: " + combinacion);
79     print("Tabla DP: ");
80     println(Arrays.toString(resultado.dp));
81 }
82
83 public static void main(String[] args) {
84     println(" SOLUCION: Cambio Minimo de Monedas (PD)");
85
86     ejecutarCaso(new int[]{1, 3, 4}, 6, "1");
87
88     ejecutarCaso(new int[]{1, 2, 5}, 11, "2");
89
90     Scanner scanner = new Scanner(System.in);
91     try {
92
93         print("Ingrese las denominaciones de monedas separadas por comas (ej. 1,3,4): ");
94         String entradaMonedas = scanner.nextLine();
95
96         String[] partes = entradaMonedas.split(",");
97         int[] monedasUsuario = new int[partes.length];
98         for (int i = 0; i < partes.length; i++) {
99             monedasUsuario[i] = Integer.parseInt(partes[i].trim());
100         }
101
102         print("ingrese la cantidad (monto) que desea analizar: ");
103         int cantidadUsuario = scanner.nextInt();
104
105         if (cantidadUsuario < 0) {
106             println("La cantidad debe ser un numero entero no negativo.");
107         } else {
108             ejecutarCaso(monedasUsuario, cantidadUsuario, "Usuario");
109         }
110
111     } catch (NumberFormatException e) {
112         println("Error: ingresar unicamente numeros enteros");
113     } catch (Exception e) {
114         println("Ocurrio un error al leer los datos de la terminal.");
115     } finally {
116         scanner.close();
117     }
118
119     println("-----");
120     println("Ejecucion finalizada.");
121 }
122 }

```

Listing 2: Implementación en Java del Cambio Mínimo de Monedas

2.7. Evidencias de Ejecución

Se ejecutaron pruebas unitarias automatizadas y pruebas personalizadas mediante el scanner. La captura de pantalla en la figura 2 valida el comportamiento correcto y la visualización de la traza de monedas.

```
diego {at-symbol-color} archlinux

^ OS      → Arch Linux x86_64
^ Kernel  → Linux 7.0.12-arch1-1
^ Uptime  → 1 hour, 26 mins
^ PKGS    → 11 (flatpak), 1145 (pacman)

^ WM      → Hyprland 0.55.4 (Wayland)
^ Shell   → zsh 5.9.1
^ Term    → kitty 0.47.1
^ Font    → Fira Code Medium (14pt, Regular) [Qt], Fira Code (14pt, SemiBold) [GTK3]

^ CPU     → Intel(R) Core(TM) i5-8265U (8) @ 3.90 GHz
^ RAM     → 5.56 GiB / 7.60 GiB (73%)
^ Bateria → 100% [AC Connected]

diego@archlinux in ~
$ cd ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/
$ ls
AlgoritmosVoracesActividad08  Ejercicio01  ProgramacionDinamicaActividad09
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos on main x (origin/main)
$ cd ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09 on main x (origin/main)
$ ls
CambioMonedas.java  latex  README.md
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09 on main x (origin/main)
$ java CambioMonedas.java
=====
SOLUCION: Cambio Minimo de Monedas (PD)
=====

Caso de prueba 1: Monedas = [1, 3, 4], Cantidad = 6
Cantidad minima de monedas: 2
Combinacion: 3+3
Tabla DP: [0, 1, 2, 1, 1, 2, 2]

-----
Caso de prueba 2: Monedas = [1, 2, 5], Cantidad = 11
Cantidad minima de monedas: 3
Combinacion: 1+5+5
Tabla DP: [0, 1, 1, 2, 1, 2, 1, 2, 2, 3, 2, 3]

-----
Ejecucion finalizada.
diego@archlinux in ~/Documentos/AnalisisDiseñoAlgoritmos/Analisis-y-Diseño-de-Algoritmos/ProgramacionDinamicaActividad09 on main x (origin/main)
$
```

Figura 2: Captura de pantalla de la ejecución que muestra la solución correcta y la trazabilidad (combinación de monedas empleadas).